analysis

May 15, 2026

[1684]:
```python
import polars as pl
from pathlib import Path
import altair as alt

DATA_DIR = Path("data")
# Default rolling window size for charts
WINDOW_SIZE = 10

# Baselines from analysis.md
BAYESIAN_BASELINE_TURNS = 47.64
HUNT_BASELINE_TURNS = 55.16

# Enable VegaFusion for Altair to speed up rendering of large datasets
_ = alt.data_transformers.enable("vegafusion")
```

---

1 Charting Utils

[1685]:
```python
def make_chart(
    df: pl.DataFrame,
    y_col: str,
    window_size: int = WINDOW_SIZE,
    arg_col: str = "step",
    scale_x: bool = True,
    stroke: float = 2,
) -> alt.Chart | alt.LayerChart | alt.FacetChart:
    df = df.with_columns(
        pl.col(y_col).rolling_mean(window_size=window_size).
 ↪alias(f"{y_col}_rolling"),
    )
    window_size = max(window_size, 1)
    # Chart size calculations
    x_max = df[arg_col].max() * 1.05
    x_axis = alt.Axis()
    if scale_x:
        x_axis = alt.Axis(labelExpr="datum.value / 1000 + 'k'")
```

```
    x_scale = alt.Scale(domainMax=x_max)

    def format_label(label: str) -> str:
        def _cap(r: str) -> str:
            return " ".join([s.capitalize() for s in label.split(r)])

        if "_" in label:
            return _cap("_")
        elif "-" in label:
            return _cap("-")
        return label.capitalize()

    y_col_label = format_label(y_col)
    x_col_label = format_label(arg_col)

    # Define color scale for lines and rules
    series_name = "Rolling avg" if window_size > 1 else "Avg"
    y_label = (
        f"{y_col_label} ({window_size}-{x_col_label} rolling avg)"
        if window_size > 1
        else y_col_label
    )
    # Rolling average line
    line = (
        alt.Chart(df.with_columns(pl.lit(series_name).alias("series")))
        .mark_line(strokeWidth=stroke, color="steelblue")
        .encode(
            x=alt.X(arg_col, title=x_col_label, axis=x_axis, scale=x_scale),
            y=alt.Y(
                f"{y_col}_rolling",
                title=y_label,
            ),
        )
        .properties(
            title=f"{y_col_label} v {x_col_label}",
            width=500,
            height=300,
        )
    )

    return line
```

[1686]:
```
def make_dual_line(
    df,
    col1,
    col2,
    y_title,
```

```
    chart_title,
    window=WINDOW_SIZE,
    stroke: float = 1,
):
    """Rolling avg line chart comparing agent vs player on a single metric."""
    rolling = (
        df.select(["game", col1, col2])
        .with_columns(
            [
                pl.col(col1).rolling_mean(window_size=window).
↪alias(f"{col1}_r"),
                pl.col(col2).rolling_mean(window_size=window).
↪alias(f"{col2}_r"),
            ]
        )
        .select(["game", f"{col1}_r", f"{col2}_r"])
        .unpivot(
            index="game",
            on=[f"{col1}_r", f"{col2}_r"],
            variable_name="side",
            value_name="value",
        )
        .with_columns(
            pl.col("side").replace({f"{col1}_r": "Agent", f"{col2}_r":␣
↪"Player"})
        )
    )
    return (
        alt.Chart(rolling)
        .mark_line(strokeWidth=stroke)
        .encode(
            x=alt.X("game:Q", title="Game"),
            y=alt.Y("value:Q", title=f"{y_title} ({window}-game rolling avg)"),
            color=alt.Color(
                "side:N",
                scale=alt.Scale(
                    domain=["Agent", "Player"], range=["steelblue", "coral"]
                ),
                legend=alt.Legend(title=""),
            ),
        )
        .properties(title=chart_title, width=500, height=300)
    )
```

1.1 Rules for Baselines

[1687]:
```
bayes_baseline_rule = (
    alt.Chart(
        pl.DataFrame({"y": [BAYESIAN_BASELINE_TURNS], "series": ["Bayes␣
 ↪target"]})
    )
    .mark_rule(strokeDash=[4, 2], color="red")
    .encode(
        y=alt.Y("y:Q"),
        color=alt.Color(
            "series:N",
            scale=alt.Scale(domain=["Bayes target"], range=["red"]),
            legend=alt.Legend(title=""),
        ),
    )
)

hunt_baseline_rule = (
    alt.Chart(pl.DataFrame({"y": [HUNT_BASELINE_TURNS], "series": ["Hunt␣
 ↪target"]}))
    .mark_rule(strokeDash=[4, 2], color="orange")
    .encode(
        y=alt.Y("y:Q"),
        color=alt.Color(
            "series:N",
            scale=alt.Scale(domain=["Hunt target"], range=["orange"]),
            legend=alt.Legend(title=""),
        ),
    )
)
```

---

2 Q-Training Data

[1688]:
```
MAKE_Q_TRAIN_CSV = False
latest_q_train_log = max(
    Path("logs").glob("q_train_*.log"), key=lambda x: x.stat().st_mtime
)
q_train_log_file = latest_q_train_log
q_train_csv = DATA_DIR / "q_train_log.csv"

if MAKE_Q_TRAIN_CSV:
    with open(q_train_log_file, "r") as f:
        lines = [line.split("INFO")[1].strip() for line in f if "ep=" in line]
        data = []
```

```
        for line in lines:
            parts = line.split()
            ep = int(parts[0].split("=")[1])
            eps = float(parts[1].split("=")[1])
            steps = int(parts[2].split("=")[1])
            mean_turns = float(parts[3].split("=")[1])
            data.append((ep, eps, steps, mean_turns))
        q_train_df = pl.DataFrame(
            data, schema=["episode", "epsilon", "steps", "mean_turns"],␣
↪orient="row"
        )
    q_train_df.write_csv(q_train_csv)
```

3 Q-Agent Data

[1689]:
```
MAKE_Q_AGENT_CSV = False
latest_q_agent_log = max(
    Path("../game/logs").glob("game_*.log"), key=lambda x: x.stat().st_mtime
)
q_agent_log_file = latest_q_agent_log
q_agent_csv = DATA_DIR / "q_agent_log.csv"
df_schema = {
    "game": pl.Int64,
    "agent_won": pl.Boolean,
    "player_won": pl.Boolean,
    "agent_sunk": pl.Int64,
    "player_sunk": pl.Int64,
    "agent_hit": pl.Int64,
    "player_hit": pl.Int64,
    "agent_hit_diff": pl.Float64,
    "agent_sink_diff": pl.Float64,
    "turns": pl.Int64,
}
df = pl.DataFrame(schema=df_schema)

if MAKE_Q_AGENT_CSV:
    with open(q_agent_log_file, "r") as f:
        lines = [line.split("INFO")[1].strip() for line in f if "INFO" in line]
        i = 0
        game_num = 0
        while i < len(lines):
            game_data = {}

            line = lines[i]
            if not line.startswith("GAME OVER"):
                i += 1
```

```
                continue

            game_num += 1
            parts = line.split("|")
            winner_str = parts[0].split("Winner:")[1].strip()
            turns = int(parts[1].split("Turns:")[1].strip())
            game_data["agent_won"] = winner_str == "Agent"
            game_data["player_won"] = winner_str == "Player"
            game_data["turns"] = turns
            game_data["game"] = game_num
            i += 1

            line = lines[i]
            if not line.strip().startswith("Player"):
                i += 1
                continue

            parts = line.split("-")[1].split(",")
            sunk = int(parts[0].strip().split(" ")[1].split(",")[0].strip())
            hit = int(parts[1].strip().split(" ")[1].strip())
            game_data["player_sunk"] = sunk
            game_data["player_hit"] = hit
            i += 1

            line = lines[i]
            if not line.strip().startswith("Agent"):
                i += 1
                continue

            parts = line.split("-")[1].split(",")
            sunk = int(parts[0].strip().split(" ")[1].split(",")[0].strip())
            hit = int(parts[1].strip().split(" ")[1].strip())
            game_data["agent_sunk"] = sunk
            game_data["agent_hit"] = hit
            i += 1

            # Normalized differences for hits and sunk (max hits = 17, max sunk␣
↪= 5)
            agent_hit_diff = (game_data["agent_hit"] - game_data["player_hit"])␣
↪/ 17
            agent_sink_diff = (game_data["agent_sunk"] -␣
↪game_data["player_sunk"]) / 5

            df = df.extend(
                pl.DataFrame(
                    schema=df_schema,
                    data={
```

```
                        "game": game_data["game"],
                        "agent_won": game_data["agent_won"],
                        "player_won": game_data["player_won"],
                        "agent_sunk": game_data["agent_sunk"],
                        "player_sunk": game_data["player_sunk"],
                        "agent_hit": game_data["agent_hit"],
                        "player_hit": game_data["player_hit"],
                        "agent_hit_diff": agent_hit_diff,
                        "agent_sink_diff": agent_sink_diff,
                        "turns": game_data["turns"],
                    },
                )
            )
    df.write_csv(q_agent_csv)
```

---

4 Q-Training Analysis

[1690]:
```
q_learning_df = pl.read_csv(q_train_csv)
q_learning_df.show()
```

[1691]:
```
mean_turns = make_chart(q_learning_df, "mean_turns", arg_col="episode",␣
 ↪window_size=10)

mean_turns += bayes_baseline_rule + hunt_baseline_rule

# Summary Stats
print(f"Total episodes:  {q_learning_df['episode'].max()}")
print(f"Total steps:     {q_learning_df['steps'].max()}")
print(f"Best mean turns: {q_learning_df['mean_turns'].min():.1f}")
title = alt.TitleParams("Q-Learning Training - Mean Turns", anchor="middle")
(mean_turns).resolve_scale(color="independent").properties(title=title).show()
```

```
Total episodes:  40000
Total steps:     1900124
Best mean turns: 44.5
```

```
alt.LayerChart(…)
```

---

5 Q-Agent Testing Analysis

[1692]:
```
q_agent_df = pl.read_csv(q_agent_csv)
q_agent_df.show()
```

[1693]:
```
GAME_WINDOW = 100

n_games = len(q_agent_df)
agent_wr = float(q_agent_df["agent_won"].mean()) * 100
player_wr = float(q_agent_df["player_won"].mean()) * 100
avg_turns = float(q_agent_df["turns"].mean())
avg_agent_hits = float(q_agent_df["agent_hit"].mean())
avg_player_hits = float(q_agent_df["player_hit"].mean())
avg_agent_sunk = float(q_agent_df["agent_sunk"].mean())
avg_player_sunk = float(q_agent_df["player_sunk"].mean())

print(f"Games:            {n_games}")
print(f"Agent win rate:   {agent_wr:.1f}%  |  Player win rate: {player_wr:.
 ↪1f}%")
print(f"Avg turns/game:   {avg_turns:.1f}")
print(
    f"Avg agent hits:   {avg_agent_hits:.1f}  |  Avg player hits:␣
 ↪{avg_player_hits:.1f}"
)
print(
    f"Avg agent sinks:  {avg_agent_sunk:.1f}  |  Avg player sinks:␣
 ↪{avg_player_sunk:.1f}"
)

rolling = q_agent_df.with_columns(
    [
        (pl.col("agent_won").cast(pl.Float64) * 100)
        .rolling_mean(window_size=GAME_WINDOW)
        .alias("agent_win_rate")
    ]
)

win_long = (
    rolling.select(["game", "agent_win_rate"])
    .unpivot(
        index="game",
        on=["agent_win_rate"],
        variable_name="side",
        value_name="win_rate",
    )
    .with_columns(pl.col("side").replace({"agent_win_rate": "Agent"}))
)

rolling_chart = make_chart(
    win_long,
    "win_rate",
    arg_col="game",
```

```
    window_size=GAME_WINDOW,
    stroke=0.5,
)

color_scale = alt.Scale(domain=["Agent", "Player"], range=["steelblue",␣
 ↪"coral"])

win_summary = pl.DataFrame(
    {"side": ["Agent", "Player"], "win_rate": [agent_wr, player_wr]}
)
bar = (
    alt.Chart(win_summary)
    .mark_bar(size=50)
    .encode(
        x=alt.X("side:N", title=""),
        y=alt.Y("win_rate:Q", title="Win Rate (%)", scale=alt.Scale(domain=[0,␣
 ↪100])),
        color=alt.Color("side:N", scale=color_scale, legend=None),
    )
    .properties(title="Overall Win Rate", width=180, height=300)
)
bar_text = bar.mark_text(dy=-12, fontSize=13).encode(
    text=alt.Text("win_rate:Q", format=".1f")
)

title = alt.TitleParams("Q-Agent - Win Rates", anchor="middle")
((rolling_chart) | (bar + bar_text)).resolve_scale(
    y="independent", color="independent"
).properties(title=title)
```

```
Games:            10000
Agent win rate:   54.1%  |  Player win rate: 45.9%
Avg turns/game:   39.3
Avg agent hits:   15.3  |  Avg player hits: 14.7
Avg agent sinks:  4.3  |  Avg player sinks: 4.1
```

[1693]:
```
alt.HConcatChart(…)
```

[1694]:
```
STAT_WINDOW = 1000

zero_rule = (
    alt.Chart(pl.DataFrame({"y": [0]}))
    .mark_rule(strokeDash=[4, 4], color="red")
    .encode(y="y:Q")
)

turns_chart = make_chart(
```

```
    q_agent_df,
    "turns",
    arg_col="game",
    window_size=STAT_WINDOW,
    stroke=0.5,
)
hits_chart = (
    make_chart(
        q_agent_df,
        "agent_hit_diff",
        arg_col="game",
        window_size=STAT_WINDOW,
        stroke=0.5,
    )
    + zero_rule
)
sinks_chart = (
    make_chart(
        q_agent_df,
        "agent_sink_diff",
        arg_col="game",
        window_size=STAT_WINDOW,
        stroke=0.5,
    )
    + zero_rule
)

turns_chart.show()

title = alt.TitleParams("Q-Agent - Targeting Statistics", anchor="middle")
(hits_chart | sinks_chart).resolve_scale(y="independent").
 ↪properties(title=title)
```

```
alt.Chart(…)
```

[1694]:
```
alt.HConcatChart(…)
```

[1695]:
```
# 1. Turns per game distribution
turns_hist = (
    alt.Chart(q_agent_df)
    .mark_bar()
    .encode(
        x=alt.X("turns:Q", title="Total Turns", bin=alt.Bin(step=5)),
        y=alt.Y("count():Q", title="Games"),
        color=alt.value("steelblue"),
    )
    .properties(title="Turns per Game Distribution", width=300, height=250)
)
```

```
# 2. Agent turns on wins vs losses (strip + mean tick)
outcome_df = q_agent_df.with_columns(
    pl.when(pl.col("agent_won"))
    .then(pl.lit("Agent Win"))
    .otherwise(pl.lit("Agent Loss"))
    .alias("outcome")
)
strip = (
    alt.Chart(outcome_df)
    .mark_circle(opacity=0.4, size=30)
    .encode(
        x=alt.X("outcome:N", title=""),
        y=alt.Y("turns:Q", title="Total Turns"),
        color=alt.Color(
            "outcome:N",
            scale=alt.Scale(
                domain=["Agent Win", "Agent Loss"], range=["steelblue", "coral"]
            ),
            legend=None,
        ),
    )
    .properties(title="Agent Turns: Wins vs Losses", width=220, height=250)
)
mean_tick = (
    alt.Chart(outcome_df)
    .mark_tick(thickness=3, size=40, color="black")
    .encode(x=alt.X("outcome:N"), y=alt.Y("mean(turns):Q"))
)
turns_outcome = strip + mean_tick

# 3. Hit accuracy per game (hits / total turns, rolling)
acc_df = q_agent_df.with_columns(
    [
        (pl.col("agent_hit") / pl.col("turns") * 100).alias("agent_acc"),
        (pl.col("player_hit") / pl.col("turns") * 100).alias("player_acc"),
    ]
)
acc_chart = make_dual_line(
    acc_df,
    "agent_acc",
    "player_acc",
    "Hit Accuracy (%)",
    "Hit Accuracy (hits/turns %)",
    window=STAT_WINDOW,
    stroke=0.5,
)
```

```
title = alt.TitleParams("Q-Agent - Additional Analysis", anchor="middle")
(turns_hist | turns_outcome | acc_chart).resolve_scale(
    y="independent", color="independent"
).properties(title=title)
```

[1695]: alt.HConcatChart(…)